

Customer Problem

Using AWS SERVERLESS & AI SERVICES

Customer: Educational Institution

An educational institution is overwhelmed by a high volume of student academic queries, leading to delayed response times and strained manual support resources. They currently lack an automated system to provide immediate answers or a centralized database to track frequently asked questions. To resolve this, they urgently need to implement a cloud-based, AI-Powered Student Support System utilizing a serverless AWS architecture. This automated solution will expose REST API endpoints via Amazon API Gateway and use AWS Lambda to process incoming student questions. The system will integrate with an external AI service, such as the OpenAI API, to generate immediate academic responses, store all queries and answers durably in Amazon DynamoDB, and streamline future updates through a structured GitHub Actions CI/CD pipeline.

AI-Powered Student Support System

A SERVERLESS API

Objectives

Build a cloud-based API that allows students to ask academic questions and receive AI-generated responses, while tracking issues using project management tools.

You are free to explore additional tools to enhance the project while producing the expected output.

Project Architecture

Architecture Overview

AWS Cloud

CI/CD Pipeline (GitHub Actions)

API Gateway (Public API Endpoints)

Trello/Jira

Amazon API Gateway

Receives Requests

AWS Lambda

Processes requests

Amazon DynamoDB

Store questions and responses

AI Service

Call external API/Use simple ML model

Phased Milestones

Phase 1: Project Setup and Planning

Activities

- Set up Trello/Jira board: backlog, sprints, roles (DevOps, Backend, QA)
- Create GitHub repo, branch strategy (main/dev/feature), and README
- Provision AWS account, enable free-tier services, configure IAM roles

Phased Milestones

Phase 2: API Development

Challenge: Build the 4 core REST API endpoints

Activities

POST	/ask	Ask questions
GET	/question	View previous questions
GET	/FAQ	View commonly asked questions/ enquiries
DELETE	/question/{id}	

- Research DynamoDB table design for questions and enquiries
- Implement robust error handling and logging
- Learn about CORS for web integration
- Write unit tests for lambda handlers

Phased Milestones

Phase 3: AI integration

Challenge: Integrate AI response layer with OpenAI API or keyword classifier in python

Activities

- Connect Lambda to AI service: pass question -> receive answer -> store in DynamoDB
- Add input validation and error handling
- Test AI responses manually with sample academic questions

Phased Milestones

Phase 4: CI/CD pipeline

Challenge: Create a Workflow

Activities

- Create GitHub Actions workflow: trigger on push to dev/main branches
- Pipeline stages: lint -> run tests -> package Lambda -> deploy to AWS
- Store credentials securely (eg Github Secrets)
- Validate end to end: push code -> auto deploy -> verify live API response

Phased Milestones

Phase 5: Logging and Monitoring

Activities

- Enable AWS CloudWatch: Lambda execution logs, API Gateway access logs
- Set up CloudWatch alarm for Lambda errors and latency thresholds
- Apply IAM least privilege to Lambda execution role and DynamoDB access

Deliverables:

- GitHub repo with API code
- CI/CD pipeline (GitHub Actions)
- Lambda functions
- DynamoDB table definitions
- CloudWatch alarms config
- README file
- Product presentation (problem, challenges, demo)

Phased Milestones

Phase 5: Deployment and Optimization

Activities

- Research deployment automation tools and techniques
- Investigate monitoring and logging solutions
- Learn about cost optimization strategies for cloud resources
- Understand how to set up proper resource lifecycle policies
- Implement automated deployment scripts

Deliverables:

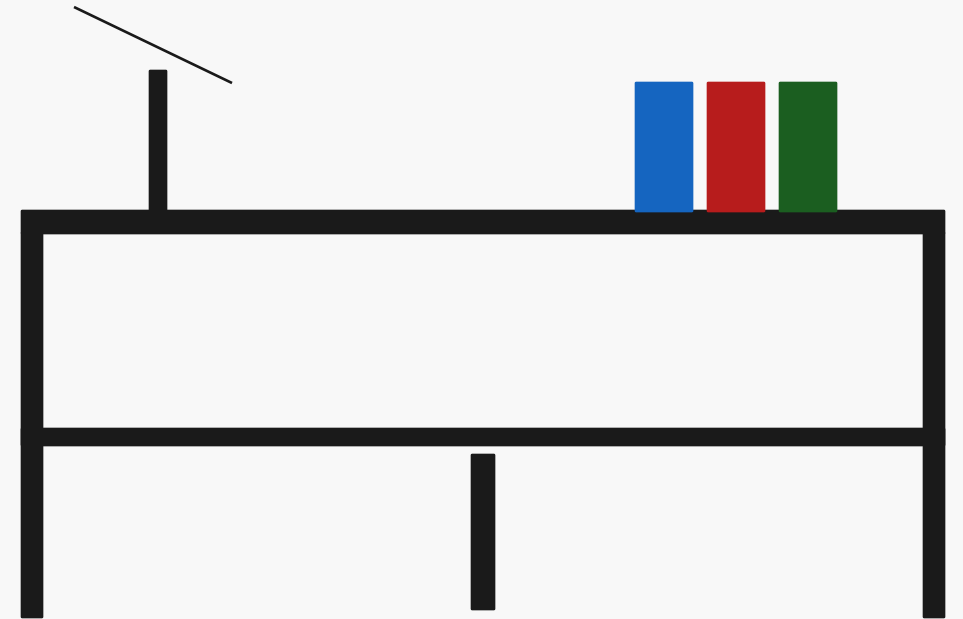
- GitHub repo with API code
- CI/CD pipeline (GitHub Actions)
- Lambda functions
- DynamoDB table definitions
- CloudWatch alarms config
- README file
- Product presentation (problem, challenges, demo)

Self-study Material Links

Links to videos, resources and documentation

For quick references and guides, here are introductory and practical resources to assist you along the way.

- [GitHub Actions Documentation](#)
- [AWS Lambda Documentation](#)
- [Amazon API Gateway](#)
- [Amazon DynamoDB](#)
- [Amazon CloudWatch](#)
- [AWS Budgets](#)
- [Boto3 \(AWS SDK for Python\)](#)
- [AWS CloudFormation](#)
- [OpenAI API Documentation](#)



Links to Resources and Documentation

Links to videos, resources and documentation

Documentation for tools used in this project

GitHub Actions

<https://docs.github.com/en/actions>

AWS Lambda

<https://docs.aws.amazon.com/lambda>

API Gateway

<https://docs.aws.amazon.com/apigateway>

DynamoDB

<https://docs.aws.amazon.com/dynamodb>

CloudWatch

<https://docs.aws.amazon.com/cloudwatch>

IAM

<https://docs.aws.amazon.com/iam>

OpenAI

[OpenAI API Platform Documentation](#)

Remember: The goal is not just to build the application, but to understand the why behind each architectural decision.

Focus on learning the principles that will apply to future projects!